Unit No.4-tree

Title:AVL Tree Traversal System with Balance Factor Analysis and Sorted Order Verification Using C++

1. Problem Statement

Implement an AVL Tree Traversal System using C++ to construct a self-balancing binary search tree from user-entered integers. The system should perform inorder, preorder, postorder, and level-order traversals while displaying the balance factor of each node. It should also allow users to insert additional values, automatically update the tree structure and traversals, and verify whether the inorder traversal is in sorted order by displaying PASS or FAIL.

2. Objectives

- To implement a self-balancing AVL tree using C++.
- To insert user-entered integers and maintain tree balance using rotations.
- To perform inorder, preorder, postorder, and level-order traversals.
- To display the balance factor of each node during traversal.
- To allow additional values to be inserted into the existing AVL tree.
- To update the tree traversals after every new insertion.
- To verify whether the inorder traversal is sorted and display PASS or FAIL.
- To understand AVL tree operations and balancing techniques.

3. Algorithm

Step 1: Start the program.

Step 2: Initialize the AVL tree root as NULL.

Step 3: Accept integers from the user and insert them into the AVL tree.

Step 4: Calculate the height and balance factor of each node.

Step 5: Perform the appropriate AVL rotation if the tree becomes unbalanced.

Step 6: Display a menu containing the following options:

Display all traversals.

Insert additional values.

Verify inorder sorted order.

Exit.

Step 7: If the user selects Display Traversals, perform inorder, preorder, postorder, and level-order traversals and display each node with its balance factor.

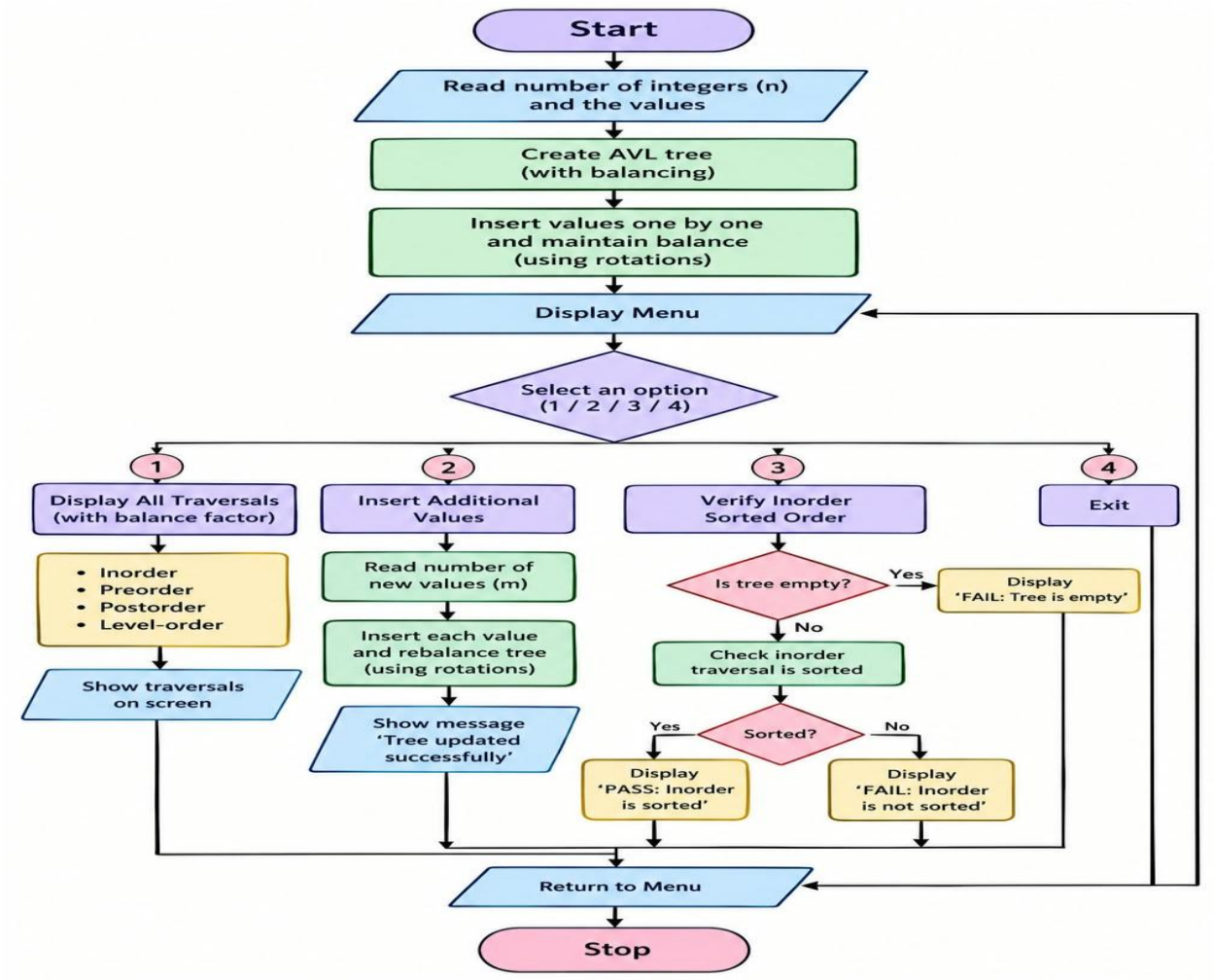
Step 8: If the user selects Insert Additional Values, accept new integers, insert them into the existing AVL tree, and rebalance the tree whenever required.

Step 9: If the user selects Verify Inorder, check whether the inorder traversal is sorted. Display PASS if sorted; otherwise, display FAIL.

Step 10: Repeat the menu until the user selects Exit.

Step 11: Stop the program.

4.Flowchart



5.Conclusion

The AVL Tree Traversal System was successfully implemented using C++. The application performs inorder, preorder, postorder, and level-order traversals and displays the balance factor of each node. It allows users to insert additional values while maintaining the balance of the tree using rotations. The system also verifies whether the inorder traversal is sorted and displays PASS or FAIL. Thus, this application helps in understanding AVL tree operations, balancing techniques, and traversal methods effectively.